

Activity Guide — Schedule a Daily Overwriting Query

Guide to the Day 1 closing demo (4:20–4:30 PM CT): five minutes, one click — a query on a daily schedule that rebuilds a derived table. Watch it in class, then replicate it after class with the SQL file in this folder. Idea from the Do-It-Now (DIN) activities page, §29 "Exploring derived tables," step 5.

What you'll do

See — and later repeat — the simplest automation on Google Cloud: take an aggregation your analysts run a hundred times a day, materialize it once into a derived table, and schedule the query to rebuild that table every day at 12:01 AM with write preference **overwrite**.

Why it matters for your migration

- The derived table is the cost story in miniature: same answer, a fraction of the bytes scanned. That is the entire case for your **Gold-layer data marts** feeding Power BI instead of letting reports hammer the base tables.
- The scheduling click is your **nightly SQL Agent job, translated**: a SQL statement, on a schedule, overwriting a table. No scheduler box, no agent service, no SSIS package.
- It is also the honest bridge to Day 2: hand-rolled derived tables go stale and multiply. Tomorrow, Dataform-managed marts replace them — same idea, governed.

How to schedule a query in the BigQuery console

1. Open **BigQuery Studio**, paste your query into the editor, and **run it once** to confirm it works.
2. Above the editor, click **Schedule → Create new scheduled query**.
3. Give it a name and set it to repeat **daily at 12:01 AM** (the DIN §29 step 5 schedule).
4. For the destination, choose your dataset — your class dataset from this morning (DIN §3) — and a table name, for example `march_zip_sales`.
5. Set the write preference to **Overwrite table** (`WRITE_TRUNCATE`).
6. Save. **The first schedule in a project pops a one-time Google authorization** for the BigQuery Data Transfer Service (scheduled queries run through it) — approve it, and enable the API if asked. If nothing happens when you click Save, look for a blocked-popup icon at the right end of the address bar: allow popups for `console.cloud.google.com` and click Save again. Once saved, the job appears under **Scheduled queries** in the BigQuery console — open it and use the manual-run option once to prove it fires.

What "overwrite destination" means

Every run replaces the table's entire contents — `TRUNCATE + INSERT` as one habit. The table is rebuilt from scratch each night, so re-runs are idempotent: no duplicates, no merge logic. (The

alternative, **Append**, adds rows on top — right for accumulating history, wrong for a rebuilt snapshot.)

Why this is the cheapest automation in GCP

A scheduled query is serverless: nothing to provision, patch, or babysit. You write SQL; BigQuery runs it on the clock. It is the floor of the automation ladder — the [M5/M6 notes](#) show the rungs above it (Cloud Scheduler, Dataform release configurations, Composer).

Try it after class

The file `nested_queries.sql` in this folder — unchanged from the course demo repository — holds the example queries the DIN §29 activity is built around: March 2018 sales-by-zip queries against the shared nested/repeated demo table.

1. Open the file and take the first query (sales by zip for March, from the `nested_once` table).
2. Replace the `<project-id>` placeholder with the shared demo project — your instructor will confirm it in class. For this delivery it is **roigcp-imran-ahmad** (dataset `bq_demo`), the instructor's demo project — the DIN page's own `roi-bq-demos` copy is not reachable from our accounts, so the instructor rebuilt the same tables at class size with `build_demo_tables.sql` (same schema, same queries, ~1.2M orders instead of billions).
3. Run it once in your lab project, writing results into your class dataset (destination `class.march_zip_sales`).
4. Schedule it using the steps above: daily, 12:01 AM, overwrite.

Two things to notice while you run it:

- `UNNEST(line_items)` — the demo table is nested/repeated: one order row carries an array of line-item structs. Different from your flat SQL Server tables, and one of BigQuery's genuinely new tricks worth a second look.
- **Bytes processed** (the editor shows it before and after a run). The full DIN §29 activity compares querying the base table against the small derived one — glance at the scan size of each; the derived table is tiny by construction. That contrast is your Power BI cost argument. The scheduled overwrite itself is low cost.

Common gotchas

- **API prompt:** scheduled queries ride on the BigQuery Data Transfer Service. If the console asks to enable it, click Enable and continue.
- **Consent popup:** the first scheduled query you ever save in a project opens a Google authorization window for the Data Transfer Service. It is a popup — if your browser blocked it, nothing happens on Save and there is no error. Allow the popup and Save again.
- **Destination dataset must exist first.** Your class dataset does (you created it this morning); any other dataset must be created before a schedule can target it.

- **Overwrite vs append:** double-check the write preference. An accidental append doubles the table every night.
- **Project/location mix-ups:** while learning, keep the source data, the destination dataset, and the schedule in your lab project and its default location — cross-project and cross-location combinations are where first attempts go wrong.
- **It runs as you by default:** fine in a lab project. At work you would run it as a service account — your SQL Agent proxy instinct is the right one; keep it.

Go further (the rest of DIN §29)

The full activity (steps 2–4, about 20 minutes) has you build the derived table, then time the same aggregation against the full table and against the derived table, comparing duration and bytes processed. That arms you with real numbers for the "why Gold marts" conversation back home. The DIN page also asks the right skeptical questions — derived tables go stale and duplicate data — which is exactly why Day 2 replaces hand-rolled derived tables with Dataform-managed marts, and where partitioning, clustering, and materialized views enter the conversation.

Self-check

- ☐ I can find **Schedule** → **Create new scheduled query** in BigQuery Studio.
- ☐ I can explain what overwrite (WRITE_TRUNCATE) does, and when I would choose append instead.
- ☐ I have a job visible under **Scheduled queries** and have seen it run once.
- ☐ I can name the SQL Agent equivalent of every part of this setup (job step = the query, schedule = daily 12:01 AM, destination = the derived table).
- ☐ I can explain to a Power BI teammate why querying the derived table is cheaper than hitting the base table.

What this replaces in your current stack

The nightly SQL Agent job that runs T-SQL to rebuild a reporting table — minus the job engine, the proxy account wrangling, and the server it ran on. Keep the instinct ("materialize what analysts hammer, refresh it on a schedule"); the platform absorbed the plumbing.

- The Do-It-Now (DIN) activities page:
<https://roittraining.github.io/gcp-demos/bigquery.html> (§29, step 5; your class dataset is from §3)
- Docs: [Scheduling queries in BigQuery](#)
- Example SQL: nested_queries.sql

Part of the Iowa DOM DoIT Data Engineering student pack — see ../00_schedule.md.